# eCoach Backend Implementation Plan — Canonical Synthesis

Best-of-both consolidation of Agent 1 and Agent 2

March 29, 2026

## Status

**Document type:** Canonical backend implementation plan
**Audience:** backend engineers, data/model engineers, technical leads, content-platform engineers
**Purpose:** provide the single implementation document that should replace the two competing plans and remove the ambiguities that would otherwise create schema churn, ownership conflicts, and architecture debt.

This document deliberately combines:

- **Agent 2's strengths:** architecture, canonical truth ownership, runtime/foundry separation, recompute discipline, governance, and phased sequencing.
- **Agent 1's strengths:** concrete schema rollout, formulas, Tauri command surface, runtime crate layout, and delivery detail.

This document is **normative**. Where it says **MUST**, the implementation is expected to follow it. Where it says **SHOULD**, deviation is allowed only with a written justification. Where it says **MAY**, the choice is optional.

## Executive summary

eCoach SHALL be implemented as **two connected modular monoliths** that share contracts but do not share databases:

1. **Learner Runtime Plane**
   - local-first
   - Rust backend
   - Tauri command boundary
   - SQLite in WAL mode
   - source of truth for learner state, session execution, diagnostics, planning, memory, reporting, local role views, and installed pack data
2. **Content Operating System / Foundry Plane**
   - central content and curriculum operations backend
   - Rust backend
   - Postgres + object storage

  - source of truth for official curriculum ingestion, source parsing, OCR/layout extraction, artifact build, review, provenance, publishing, pack compilation, and pack release metadata

The most important architectural freeze in this synthesis is:

- **one shared learner truth**
- **one shared content truth**
- **premium and elite are policy layers, not separate architectures**
- **frontend renders backend decisions; it does not own academic logic**
- **event ledgers and decision traces are first-class**
- **replayability, explainability, and provenance are mandatory**

# 1. Non-negotiable design principles

## 1.1 Platform identity

eCoach is **not** a generic AI tutor, a thin quiz shell, or a frontend-heavy learning app.

It **is**:

- a local-first academic coaching runtime
- a curriculum-aware evidence system
- a deterministic planning and intervention engine
- a content operating system with review and publishing gates
- an audit-friendly readiness and proof system

## 1.2 Decision principles

The backend MUST enforce the following rules:

1. **No mastery from one success.**
2. **No readiness without threshold evidence.**
3. **No new learning when a protection gate is active on prerequisites.**
4. **No live content without provenance, review status, and pack version traceability.**
5. **No direct cross-module writes into another module's source-of-truth tables.**
6. **No frontend-owned academic rules.**
7. **No free-form generation as runtime source of truth.**
8. **No separate learner-truth models for premium or elite.**

## 1.3 Implementation posture

The implementation SHOULD optimize for:

- deterministic recompute

- offline correctness
- small blast radius per module
- explainable policy gates
- migration stability
- developer ergonomics for Rust + SQL + replay tests

## 2. Frozen canonical decisions

### 2.1 Canonical academic units

The previous plans drifted between topic, node, skill, concept, atom, and academic node. That ambiguity is now frozen.

| Canonical unit | Meaning | Source of truth | Used by |
|---|---|---|---|
| CurriculumNode | official syllabus unit in the executable curriculum graph | Foundry | planning, coverage, reporting, packs |
| ConceptAtom | smallest teachable meaning unit | Foundry | teaching, glossary, explanations, traps |
| SkillAtom | smallest assessable performance unit | Foundry | evidence, mastery, diagnostics, readiness |
| KnowledgeUnit | umbrella interface implemented by ConceptAtom and SkillAtom | shared contract | retrieval, linking, library |

**Rules:**

- learner evidence MUST anchor to SkillAtom
- explanatory content MUST anchor to ConceptAtom
- both MUST map upward to CurriculumNode
- question items MAY target multiple SkillAtoms, but exactly one MUST be marked primary
- readiness, coverage, and plan generation MUST operate primarily at CurriculumNode and aggregate downward evidence from mapped SkillAtoms

### 2.2 Shared truth rules

| Truth family | Canonical owner |
|---|---|
| learner truth | Learner Runtime Plane |
| official curriculum and content artifacts | Foundry Plane |
| read models | derived only, never source of truth |

| Truth family | Canonical owner |
|---|---|
| feature overlays (premium/elite) | policy layer on top of the same learner truth |

## 2.3 Architecture style

| Concern | Canonical choice |
|---|---|
| runtime architecture | Rust modular monolith |
| runtime DB | SQLite, WAL mode, foreign keys ON |
| runtime shell | Tauri v2 command boundary |
| runtime jobs | in-process tokio workers |
| foundry architecture | Rust modular monolith |
| foundry DB | Postgres |
| foundry assets | object storage |
| score type | BasisPoints = u16 |
| timestamps persisted | ISO 8601 text |
| event IDs | UUID/ULID text |
| external stable IDs for pack/artifact/version | text |
| JSON columns | allowed, but validated at app boundary |

## 2.4 Basis points and score conventions

BasisPoints is frozen as:

```
pub type BasisPoints = u16; // 0..=10_000
```

This resolves the Agent 1 inconsistency between u16 and i32.

**Rules:**

- persisted score columns use integer basis points
- formulas MAY compute in f64, but MUST clamp and convert back to BasisPoints
- percentages SHOULD never be stored as floats in source-of-truth tables
- any value that can go below zero MUST use a separate signed type, not BasisPoints

## 2.5 Entitlements and access

EntitlementTier is frozen as:

```
pub enum EntitlementTier {
  Standard,
  Premium,
```

```
    Elite,
}
```

This resolves the Agent 1 Standard/Premium/Elite vs Free/Standard/Elite conflict.

If a freemium or trial mode is later introduced, it MUST be modeled separately as LicensePlan or AccessMode, not by redefining entitlement tiers.

## 2.6 Split the conflated state machines

One of the most dangerous problems in Agent 1 was that two very different state machines were mixed together under the same name. This synthesis separates them cleanly.

### A. Learner Journey State

This is the product/UX gate state that determines which shell or next screen the learner sees.

```
pub enum LearnerJourneyState {
    OnboardingRequired,
    SubjectSelectionRequired,
    DiagnosticRequired,
    ContentReadinessRequired,
    PlanGenerationRequired,
    ReadyForTodayMission,
    MissionInProgress,
    MissionReviewRequired,
    RepairRequired,
    BlockedOnTopic,
    PlanAdjustmentRequired,
    ReviewDay,
    ExamMode,
    StalledNoContent,
}
```

### B. Coach Operational Mode

This is the internal coaching mode that determines what the orchestration engine is trying to do.

```
pub enum CoachOperationalMode {
    Dormant,
    Calibrating,
    BaselineSetting,
    MissionPlanning,
    ActiveTeaching,
    GapRepair,
    MemoryRescue,
```

```
    MockOrchestrating,
    ReadinessProofing,
    CrisisIntervention,
    ExamCountdown,
    PostExamReview,
    ParentalBriefing,
    SystemMaintenance,
}
```

These are **two separate state machines**. They MUST be stored and resolved separately.

## 2.7 Mastery model

The previous plans also drifted on mastery naming and thresholds. This synthesis freezes both the state ladder and the gates.

### Canonical mastery state ladder

```
pub enum MasteryState {
    Unseen,        // no meaningful evidence
    Emerging,      // first exposures
    Unstable,      // some success, inconsistent
    Functional,    // reliable in standard conditions
    Stable,        // consistent across sessions/forms
    Transferable,  // succeeds in novel contexts
    ExamReady,     // succeeds under timed pressure
    Mastered,      // durable, comprehensive, low-fragility
}
```

### Canonical thresholds

| Threshold | Basis points | Meaning |
|---|---:|---|
| MASTERY_EMERGING_MIN | 1000 | first credible signal |
| MASTERY_UNSTABLE_MIN | 2500 | non-random but weak |
| MASTERY_FUNCTIONAL_MIN | 4000 | standard-condition reliability begins |
| MASTERY_STABLE_MIN | 5500 | session-to-session reliability |
| MASTERY_TRANSFERABLE_MIN | 7000 | transfer success |
| MASTERY_EXAM_READY_MIN | 8500 | timed exam-grade readiness |
| MASTERY_MASTERED_ | 9000 | durable top state |

| Threshold | Basis points | Meaning |
|---|---|---|
| MIN | | |

### Canonical gates

| Gate | Basis points | Purpose |
|---|---|---|
| MASTERY_PROTECTION_GATE | 4000 | below this, prerequisites may block downstream progression |
| MASTERY_DEPENDENCY_UNLOCK_GATE | 5500 | dependent topic progression unlocked at Stable-or-better |
| MASTERY_MOCK_ELIGIBILITY_GATE | 4000 | topic may appear in non-diagnostic mocks |
| MASTERY_READINESS_TOPIC_GATE | 5500 | topic counts toward readiness coverage at Stable-or-better |
| MASTERY_ELITE_TOPIC_GATE | 8500 | elite breadth gate |

This keeps the useful 4000 and 5500 gates from Agent 1, but gives them distinct meanings instead of contradictory names.

## 2.8 Memory model

Agent 1 mixed a scheduler memory ladder with a library/display ladder. This synthesis freezes the **core scheduler** state machine and treats display states as derived.

```
pub enum MemoryState {
  NotFormed,
  Fresh,
  Consolidating,
  Stable,
  AtRisk,
  Decaying,
  Critical,
  Rescued,
  Forgotten,
}
```

**Rules:**

- scheduler state uses the ladder above
- library shelf UI MAY expose derived labels like reinforced or transferred, but those are not core scheduler states

- memory review cadence and rescue logic MUST operate on the core scheduler state only

# 3. Target architecture

## 3.1 Plane model

| Plane | Primary users | Canonical responsibilities |
|---|---|---|
| Learner Runtime Plane | student, parent, local admin | learner truth, sessions, diagnostics, planning, memory, readiness, library, glossary state, reporting, local pack install |
| Content Operating System / Foundry Plane | super-admin, content ops, curriculum ops | source intake, OCR, curriculum ingestion, artifact build, provenance, review, publish, pack compile/sign/release |

## 3.2 Runtime plane responsibilities

The runtime owns:

- local accounts and role context
- installed pack registry and active curriculum slices
- question execution and attempt capture
- learner truth reducers
- diagnostics
- coach planning and mission orchestration
- memory decay and review queueing
- readiness and parent/admin read models
- local search indexes
- local backup/restore

## 3.3 Foundry plane responsibilities

The foundry owns:

- source document lifecycle
- OCR and layout extraction
- curriculum parsing and versioning
- artifact generation and comparison
- content trust, provenance, and review
- pack compilation, signing, and release manifests
- publish history and rollback metadata

### 3.4 Why modular monoliths, not microservices

This system SHOULD NOT start as microservices because:

- learner truth requires in-process low-latency coordination
- offline-first runtime is naturally one process
- replay and deterministic recompute are easier with a single local event substrate
- content governance wants strong transactional boundaries
- operational complexity would exceed actual product value at this stage

## 4. Repository and workspace structure

The workspace below fuses Agent 2's domain boundaries with Agent 1's concrete runtime crate detail.

```
apps/
  runtime-backend/
    src-tauri/
    Cargo.toml
  foundry-backend/
    Cargo.toml

crates/
  shared-types/
  shared-contracts/

  platform-storage/
  platform-events/
  platform-jobs/
  platform-search/
  platform-audit/
  platform-policy/
  platform-evaluation/

  domain-curriculum/
  domain-knowledge/
  domain-question-intelligence/
  domain-content-artifacts/

  domain-learner-truth/
  domain-diagnostics/
  domain-instruction/
  domain-memory/
  domain-planning/
```

```
  domain-session-runtime/
  domain-readiness/
  domain-library/
  domain-parent-admin-runtime/
  domain-games/

  foundry-source-intake/
  foundry-curriculum-ingestion/
  foundry-artifact-build/
  foundry-quality-governance/
  foundry-packaging-delivery/

  runtime-commands/

migrations/
  runtime/
  foundry/

packs/
  schemas/
  compiler/
  fixtures/

tests/
  unit/
  integration/
  replay/
  contracts/
  performance/
```

### 4.1 Module mapping: source plans to final modules

| Final module | Inherits mainly from |
|---|---|
| domain-learner-truth | Agent 2 architecture + Agent 1 student-model schema |
| domain-diagnostics | Agent 2 diagnostics + Agent 1 diagnostic tables |
| domain-memory | Agent 2 memory discipline + Agent 1 decay formulas and schedules |
| domain-planning | Agent 2 planning doctrine + Agent 1 coach plan/missions |
| runtime-commands | Agent 1 Tauri command surface |
| foundry-* | Agent 2 Content OS / Foundry plane |

| Final module | Inherits mainly from |
|---|---|
| platform-policy | Agent 2 governance + Agent 1 hard rules and thresholds |

# 5. Canonical domain model

## 5.1 Curriculum domain

| Entity | Purpose | Owning plane |
|---|---|---|
| CurriculumVersion | published syllabus version | Foundry |
| Subject | top-level subject | Foundry |
| Strand / SubStrand | official hierarchy | Foundry |
| CurriculumNode | executable syllabus node | Foundry |
| CurriculumRelation | prerequisite/related edges | Foundry |
| LearningObjective | objective statements and simplified variants | Foundry |
| NodeCoverageLedger | pack/artifact completeness and sufficiency | Foundry |

## 5.2 Knowledge domain

| Entity | Purpose | Owning plane |
|---|---|---|
| KnowledgeEntry | canonical glossary/concept/formula item | Foundry |
| ConceptAtom | smallest teachable unit | Foundry |
| SkillAtom | smallest assessable unit | Foundry |
| KnowledgeRelation | related/prerequisite/ contrast links | Foundry |
| KnowledgeBundle | topic or exam bundle | Foundry |
| ContrastPair | confusable concept pair | Foundry |
| ContrastEvidenceAtom | structured explanation/example/hook for a pair | Foundry |
| QuestionGlossaryLink | links questions to glossary/knowledge items | Foundry -> packaged to runtime |

## 5.3 Question intelligence domain

| Entity | Purpose | Owning plane |
|---|---|---|
| QuestionItem | canonical question | Foundry, installed into runtime |

| Entity | Purpose | Owning plane |
|---|---|---|
| QuestionOption | answer option | Foundry, installed into runtime |
| QuestionIntelligenceSnapshot | multi-axis classification | Foundry |
| QuestionFamily | recurring pattern / mutation family | Foundry |
| QuestionSkillLink | maps question to skill atoms | Foundry |
| MisconceptionPattern | wrong-model pattern | Foundry |
| DistractorIntent | why a distractor is tempting | Foundry |

## 5.4 Learner truth domain

| Entity | Purpose |
|---|---|
| LearnerProfile | runtime learner preferences and exam context |
| SkillState | mastery and evidence aggregate by SkillAtom |
| ConceptState | concept familiarity/teaching state by ConceptAtom |
| CurriculumNodeState | rollup at topic/node level for planning/reporting |
| MemoryStateRecord | decay/review scheduler state |
| PressureState | pressure collapse / timed-condition state |
| RiskAssessment | current danger zones and severity |
| ReadinessState | current readiness rollup |
| LearnerStateSnapshot | materialized snapshot for home screen and reports |

## 5.5 Diagnostics and interventions

| Entity | Purpose |
|---|---|
| DiagnosticCase | a diagnostic run or case file |
| DiagnosticHypothesis | suspected root cause |
| DiagnosticProbe | probe question / activity to test a hypothesis |
| ProblemCard | human-readable problem-cause-fix object |
| RepairPlan | repair path for a diagnosed weakness |
| InterventionPlan | broader action sequence produced by coach |

| Entity | Purpose |
|---|---|
| InterventionStep | one step in an intervention |
| ReadinessClaim | formal claim that student is ready for some scope |
| ReadinessContract | proof requirements for the claim |

## 5.6 Planning and runtime

| Entity | Purpose |
|---|---|
| CoachOperationalModeState | internal coach mode |
| LearnerJourneyStateRecord | UX gate state |
| NextCoachAction | next backend instruction to UI |
| DailyPlan | daily target plan |
| WeeklyPlan | weekly plan |
| Mission | atomic coaching mission |
| MissionItem | one piece inside a mission |
| SessionPlan | concrete launch blueprint |
| SessionObject | persisted runtime session |
| ScheduleLedger | truth of planned vs actual time |
| PresenceStateSnapshot | live/paused/abandoned activity state |

## 5.7 Library and personal knowledge

| Entity | Purpose |
|---|---|
| LibraryItem | typed saved item |
| LibraryShelf | grouping and organization |
| MistakeCase | persisted error/misconception case |
| RevisionPack | saved set of items/questions/content |
| StudentEntryState | learner's state for a glossary or knowledge entry |
| MemoryShelfItem | reviewable knowledge item, derived from core memory system |

## 5.8 Foundry domain

| Entity | Purpose |
|---|---|
| SourceDocument | uploaded source |
| SourcePage / SourceSegment | structural parse units |
| ParseJob | OCR / segmentation / mapping job |
| Artifact | generated content artifact |

| Entity | Purpose |
|---|---|
| ArtifactVersion | immutable artifact version |
| ArtifactQualityReport | quality/trust evaluation |
| ArtifactReview | review action and decision |
| ContentTrustState | trust/provenance state |
| PublishVersion | pack publish record |
| PackManifest | installable pack description |

# 6. Persistence strategy

## 6.1 Runtime storage model

The runtime uses:

- SQLite for structured tables
- local filesystem for installed packs and media
- FTS indexes for glossary/library/content search
- append-only event tables for evidence and decision traces
- materialized state tables for hot reads

## 6.2 Runtime table families

| Family | Examples |
|---|---|
| append-only ledgers | event_log, student_question_attempts, memory_evidence_events, audit_log |
| state tables | skill_states, concept_states, curriculum_node_states, memory_states, readiness_states, coach_plans |
| reference/install tables | content_packs, subjects, curriculum_nodes, questions, knowledge_entries |
| read models | home_dashboard_read_model, parent_digest_read_model, coach_directive_read_model |

## 6.3 Runtime migration rollout

The final runtime migration track preserves Agent 1's good concreteness but relocates source-side foundry concerns out of the local app.

### Runtime migrations (canonical)

| Migration | Main tables / purpose |
|---|---|
| 001_identity.sql | accounts, student_profiles, parent_profiles, parent_student_links, local_admin_profiles |
| 002_curriculum.sql | installed curriculum_versions, subjects, curriculum_nodes, curriculum_relations, learning_objectives |
| 003_knowledge.sql | knowledge_entries, knowledge_relations, knowledge_bundles, contrast_pairs, contrast_evidence_atoms, question_glossary_links |
| 004_questions.sql | question_families, questions, question_options, question_skill_links, misconception_patterns |
| 005_learner_truth.sql | skill_states, concept_states, curriculum_node_states, student_error_profiles, learner_state_snapshots |
| 006_sessions.sql | sessions, session_items, student_question_attempts, session_events, schedule_ledgers |
| 007_diagnostics.sql | diagnostic_cases, diagnostic_hypotheses, diagnostic_probes, problem_cards, diagnostic_item_attempts |
| 008_coach_planning.sql | coach_operational_modes, learner_journey_states, coach_plans, coach_plan_days, coach_missions, mission_items, coach_blockers, coach_session_evidence |
| 009_memory.sql | memory_states, memory_evidence_events, review_schedules, interference_edges, memory_shelf_items |
| 010_mock_and_forecast.sql | mock_blueprints, mock_sessions, mock_question_log, forecast_blueprints, installed past_paper_blueprint_read_models |
| 011_repair_and_gap.sql | gap_repair_plans, gap_repair_plan_items, solidification_sessions, gap_evidence_log, intervention_plans, intervention_steps |

| Migration | Main tables / purpose |
|---|---|
| 012_goals_calendar.sql | exam_goals, preparation_phases, calendar_events, study_intensity_settings, milestone_records |
| 013_library.sql | library_items, library_shelves, shelf_items, mistake_bank_entries, revision_pack_items |
| 014_glossary.sql | student_entry_states, glossary_search_index, FTS tables, glossary audio program tables |
| 015_reporting.sql | readiness_claims, readiness_proofs, danger_zones, readiness_reports, weekly_digests, memo_records |
| 016_parent_admin_runtime.sql | parent digest read models, local admin settings, device/runtime operational tables |
| 017_games_and_traps.sql | game_sessions, game_events, tug_of_war_states, mindstack_states, difference_mastery_states |
| 018_content_packs.sql | content_packs, pack_install_log, manifest/install status, pack activation |
| 019_entitlements.sql | entitlement overlays, premium strategy records, elite performance records, feature targeting |
| 020_event_log.sql | event_log, outbox_events, event_consumers, domain_event_types |
| 021_search_and_indices.sql | FTS materialization helpers, query caches, search ranking metadata |
| 022_system.sql | app_settings, feature_flags, migration_history, sync_state, background_job_records, audit_log |

### Notes on what moved out of runtime

The following concerns belong in Foundry, not the runtime source-of-truth schema:

- raw source intake
- OCR output
- source document review queues
- curriculum ingestion workflows
- artifact comparison and review
- pack publish history beyond the installed-manifest view

The runtime MAY store compact installed read models derived from these, but MUST NOT own the source workflows.

## 6.4 Foundry migration rollout

| Migration | Main tables / purpose |
|---|---|
| F001_source_intake.sql | source_documents, source_pages, source_segments, source_uploads |
| F002_parse_jobs.sql | parse_jobs, ocr_results, layout_parse_results, question_candidate_records |
| F003_curriculum_ingestion.sql | curriculum_import_runs, curriculum_candidate_nodes, curriculum_reviews, curriculum_versions |
| F004_artifact_registry.sql | artifacts, artifact_versions, artifact_dependencies, artifact_build_jobs |
| F005_quality_governance.sql | artifact_quality_reports, artifact_reviews, content_trust_states, provenance_edges |
| F006_publish_packaging.sql | publish_versions, pack_manifests, pack_release_files, release_channels, rollback_records |
| F007_foundry_jobs.sql | background queue and worker tables |
| F008_foundry_audit.sql | audit_log, review_action_log, publish_history |

## 6.5 Pack structure

Every pack MUST include:

- manifest.json
- pack ID and version
- curriculum version reference
- subject coverage
- checksum set
- signature
- build timestamp
- artifact inventory
- schema version
- compatibility constraints

Runtime install MUST be transactional:

1. verify signature
2. verify checksums

3. stage unpack
4. apply install transaction
5. activate
6. update install log
7. rollback on failure

## 6.6 Canonical key columns for highest-risk tables

The tables below are the ones most likely to become expensive to change later. Their shapes should be treated as high-discipline contracts.

### student_question_attempts

| Column | Type | Notes |
|---|---|---|
| id | integer PK | local row identity |
| student_id | FK | account |
| session_id | FK | session context |
| question_id | FK | question attempted |
| attempt_number | integer | per student-question sequence |
| started_at / submitted_at | text | ISO 8601 |
| response_time_ms | integer | nullable only if abandoned |
| selected_option_id | FK nullable | MCQ path |
| answer_text | text nullable | open response path |
| is_correct | integer | 0/1 |
| confidence_level | text nullable | sure / not_sure / guessed |
| hint_count | integer | evidence-weight input |
| changed_answer_count | integer | pressure/uncertainty signal |
| support_level | text | independent / guided / heavily_guided |
| was_timed | integer | timed-condition signal |
| was_transfer_variant | integer | transfer-context signal |
| was_retention_check | integer | delayed-recall signal |
| misconception_triggered_id | FK nullable | wrong-model signal |

| Column | Type | Notes |
|---|---|---|
| evidence_weight_bp | integer | normalized final evidence weight |
| created_at | text | immutable append time |

### skill_states

| Column | Type | Notes |
|---|---|---|
| student_id | FK | composite key part |
| skill_atom_id | FK | composite key part |
| mastery_bp | integer | canonical mastery score |
| mastery_state | text | derived band/state |
| accuracy_bp | integer | dimension |
| speed_bp | integer | dimension |
| retention_bp | integer | dimension |
| transfer_bp | integer | dimension |
| consistency_bp | integer | dimension |
| fragility_bp | integer | collapse risk |
| pressure_collapse_bp | integer | timed degradation |
| total_attempts | integer | evidence counter |
| correct_attempts | integer | evidence counter |
| last_seen_at | text | recency |
| last_mastered_at | text nullable | state milestone |
| next_review_at | text nullable | memory handoff |
| version | integer | optimistic recompute/versioning |

### coach_missions

| Column | Type | Notes |
|---|---|---|
| id | integer PK | mission identity |
| student_id | FK | owner |
| plan_day_id | FK nullable | schedule link |
| journey_state_snapshot | text | learner journey state at creation |
| operational_mode_snapshot | text | coach mode at creation |
| title | text | learner-facing title |
| reason | text | explainability text |

| Column | Type | Notes |
|---|---|---|
| subject_id | FK nullable | scope |
| primary_topic_id | FK nullable | scope |
| activity_type | text | learn, repair, mock, etc. |
| target_minutes | integer | planned effort |
| steps_json | json text | structured steps |
| success_criteria_json | json text | structured gates |
| status | text | pending/active/completed/skipped/deferred |
| created_at | text | |
| completed_at | text nullable | |

### readiness_claims

| Column | Type | Notes |
|---|---|---|
| id | integer PK | claim identity |
| student_id | FK | owner |
| claim_scope_type | text | topic/subject/full-exam/custom |
| scope_json | json text | nodes/skills in scope |
| readiness_bp | integer | current computed readiness |
| predicted_exam_score_bp | integer nullable | forecast |
| confidence_bp | integer | claim confidence |
| claim_basis_json | json text | evidence summary |
| status | text | pending/proven/failed/expired/withdrawn |
| proven_at | text nullable | proof milestone |
| failed_at | text nullable | failure milestone |
| expires_at | text nullable | staleness management |
| version | integer | registry/version coupling |

### event_log

| Column | Type | Notes |
|---|---|---|
| id | integer PK | append order |

| Column | Type | Notes |
|---|---|---|
| event_id | text unique | UUID/ULID |
| event_type | text | canonical event name |
| aggregate_type | text | domain aggregate |
| aggregate_id | text | aggregate identity |
| student_id | FK nullable | denormalized query aid |
| occurred_at | text | business time |
| recorded_at | text | persistence time |
| payload | json text | immutable payload |
| trace_id | text | end-to-end correlation |
| causation_id | text nullable | chain link |
| schema_version | integer | event schema version |
| is_processed | integer | outbox/read-model helper only |

### content_packs

| Column | Type | Notes |
|---|---|---|
| pack_id | text unique | stable pack identity |
| pack_version | text | semantic or channel version |
| curriculum_version | text | curriculum dependency |
| subject_scope_json | json text | subjects included |
| manifest_json | json text | full manifest snapshot |
| install_path | text | local filesystem path |
| status | text | installing/installed/active/failed/removed |
| installed_at | text | |
| activated_at | text nullable | |
| error_message | text nullable | rollback/debug |

### source_documents (Foundry)

| Column | Type | Notes |
|---|---|---|
| id | PK | |
| document_key | text unique | stable identity |
| document_type | text | curriculum, past paper, textbook, upload |
| subject_id | FK nullable | target subject |

| Column | Type | Notes |
|---|---|---|
| file_uri | text | object storage or source path |
| file_hash | text | dedupe/integrity |
| processing_status | text | queued -> approved/failed |
| uploaded_by | FK nullable | actor |
| created_at | text | |
| updated_at | text | |

### artifact_versions (Foundry)

| Column | Type | Notes |
|---|---|---|
| id | PK | |
| artifact_id | FK | parent artifact |
| artifact_type | text | explanation, question, bundle, formula, etc. |
| version_label | text | immutable version |
| content_uri | text | object storage payload |
| provenance_json | json text | source lineage |
| quality_report_id | FK nullable | evaluation link |
| review_status | text | draft/review/approved/rejected |
| published_in_version_id | FK nullable | publish link |
| created_at | text | |

## 7. Event model and recompute discipline

### 7.1 Canonical event envelope

```rust
pub struct DomainEvent {
  pub event_id: String,
  pub event_type: String,
  pub aggregate_type: String,
  pub aggregate_id: String,
  pub occurred_at: String,
  pub recorded_at: String,
  pub payload: serde_json::Value,
  pub trace_id: String,
  pub causation_id: Option<String>,
```

```
    pub schema_version: u16,
}
```

## 7.2 Event naming conventions

Use dot-separated lowercase names:

- identity.account_created
- identity.pin_verified
- pack.install_started
- pack.install_completed
- session.started
- session.attempt_recorded
- session.completed
- learner.skill_state_recomputed
- diagnostic.case_started
- diagnostic.hypothesis_generated
- coach.next_action_resolved
- memory.review_scheduled
- readiness.claim_evaluated
- foundry.source_uploaded
- foundry.pack_published

## 7.3 Ownership rule

No module may write another module's source-of-truth tables directly. Cross-domain updates happen through:

- command handlers
- domain events
- recompute jobs

## 7.4 Runtime hot path

submit_attempt MUST follow this flow:

1. persist raw attempt event
2. write attempt row
3. classify outcome and misconception/distractor signal
4. update evidence aggregates for affected SkillAtoms
5. recompute skill state
6. recompute curriculum-node rollups
7. recompute pressure and memory effects if relevant
8. recompute coach next action

9. update read models
10. enqueue secondary jobs as needed

### 7.5 Session completion path

complete_session MUST:

1. finalize counted-time truth
2. close open session segments
3. update plan adherence
4. compute session summary
5. recompute readiness deltas
6. recompute review queue and next coach action
7. emit parent/admin digest deltas if enabled

### 7.6 Replay model

The system uses a **hybrid event-sourced pattern**:

- append-only event ledgers
- materialized state tables
- rebuild/replay tools for deterministic regeneration

**Rules:**

- every critical reducer MUST be replay-testable
- state rebuilds MUST be idempotent
- read-model rebuilds MUST be throwaway/derivable
- event consumers MUST support bounded retry and dead-lettering

## 8. Canonical state machines

### 8.1 Learner Journey State

#### States

| State | Meaning |
|---|---|
| OnboardingRequired | profile setup incomplete |
| SubjectSelectionRequired | profile complete, no enrolled subjects |
| DiagnosticRequired | subjects chosen, no baseline diagnostic |
| ContentReadinessRequired | required packs not installed or insufficient |
| PlanGenerationRequired | baseline exists, no active plan |
| ReadyForTodayMission | mission available and no active session |
| MissionInProgress | session active |

| State | Meaning |
|---|---|
| MissionReviewRequired | session done, review not acknowledged |
| RepairRequired | critical repair gate before normal progression |
| BlockedOnTopic | all currently relevant paths blocked by prerequisites |
| PlanAdjustmentRequired | drift or adherence failure requires replan |
| ReviewDay | scheduled review-focused day |
| ExamMode | final exam phase override |
| StalledNoContent | required content unavailable |

### Key transitions

| From | To | Trigger |
|---|---|---|
| OnboardingRequired | SubjectSelectionRequired | profile complete |
| SubjectSelectionRequired | DiagnosticRequired | at least one subject chosen |
| DiagnosticRequired | ContentReadinessRequired | diagnostic requested but content missing |
| DiagnosticRequired | PlanGenerationRequired | baseline diagnostic complete and content sufficient |
| ContentReadinessRequired | PlanGenerationRequired | minimum pack sufficiency reached |
| PlanGenerationRequired | ReadyForTodayMission | plan generated |
| ReadyForTodayMission | MissionInProgress | session started |
| MissionInProgress | MissionReviewRequired | session closed |
| MissionReviewRequired | RepairRequired | post-session recompute triggers repair gate |
| MissionReviewRequired | PlanAdjustmentRequired | performance drift or missed-plan rule |
| RepairRequired | ReadyForTodayMission | repair complete and protection gate cleared |
| any state | ExamMode | exam threshold window reached |
| any relevant state | StalledNoContent | required content unavailable locally |

## 8.2 Coach Operational Mode

| Mode | When active |
|---|---|
| Dormant | no active learner context |
| Calibrating | running initial diagnostic or re-baseline |

| Mode | When active |
|---|---|
| BaselineSetting | reducing evidence into the first state map |
| MissionPlanning | building or recalculating plan |
| ActiveTeaching | normal teach/practice mode |
| GapRepair | focused remediation |
| MemoryRescue | rescue/review priority dominates |
| MockOrchestrating | selecting or analyzing mock activity |
| ReadinessProofing | proving readiness claims |
| CrisisIntervention | risk is exam-threatening |
| ExamCountdown | final countdown mode |
| PostExamReview | exam has passed; debrief mode |
| ParentalBriefing | generating parent-facing guidance |
| SystemMaintenance | background repair/reindex/recompute |

**Resolution order:**

1. CrisisIntervention
2. ExamCountdown
3. MemoryRescue
4. GapRepair
5. ReadinessProofing
6. MockOrchestrating
7. MissionPlanning
8. ActiveTeaching
9. ParentalBriefing
10. SystemMaintenance
11. Dormant

This strict precedence prevents mode flapping and hidden planner conflicts.

## 8.3 Session lifecycle

```
pub enum SessionStatus {
  Created,
  Active,
  Paused,
  Completed,
  Abandoned,
```

```
  Failed,
}
```

**Rules:**

- time only counts if session is formally closed
- abandoned sessions preserve evidence but contribute zero counted time
- a session may be resumed only from Paused
- Completed is immutable except for derived read-model rebuilds

## 8.4 Diagnostic lifecycle

```
pub enum DiagnosticStatus {
  Created,
  Phase1BroadScan,
  Phase2AdaptiveZoom,
  Phase3ConditionTesting,
  Phase4StabilityRecheck,
  Phase5ConfidenceSnapshot,
  Completed,
  Abandoned,
}
```

The one-session DNA diagnostic MAY use a compressed path, but it MUST still produce:

- problem card(s)
- root-cause hypothesis
- evidence summary
- recommended fix path

## 8.5 Memory lifecycle

| Current state | Success path | Failure path | Default next review |
|---|---|---|---|
| NotFormed | Fresh | stays NotFormed | 1 day |
| Fresh | Consolidating | AtRisk | 1 day |
| Consolidating | Stable | AtRisk | 3 days |
| Stable | stays Stable or strengthened | AtRisk | 7 days |
| AtRisk | Fresh or Consolidating | Decaying | 2 days |
| Decaying | Rescued | Critical | same day |
| Critical | Rescued | Forgotten | same day |
| Rescued | Consolidating | AtRisk | 1 day |

| Current state | Success path | Failure path | Default next review |
|---|---|---|---|
| Forgotten | Fresh after reteach | stays Forgotten | same day |

### 8.6 Foundry publish lifecycle

```
pub enum PublishStatus {
    Draft,
    Parsed,
    Mapped,
    Reviewed,
    Approved,
    Published,
    RolledBack,
    Archived,
}
```

No artifact MAY become Published without:

- provenance
- trust status
- review action
- version and dependency record
- pack build success

## 9. Scoring, formulas, thresholds, and policy engines

### 9.1 Evidence weighting

Evidence weight is one of the best pieces from Agent 1 and is retained with normalization.

```
pub fn compute_evidence_weight(
    hint_count: u32,
    support_level: SupportLevel,
    is_transfer_context: bool,
    is_delayed_recall: bool,
    is_repeat_same_day: bool,
    is_correct_but_guessed: bool,
) -> f64
```

#### Multipliers

| Signal | Multiplier |
|---|---|
| baseline | 1.00 |

| Signal | Multiplier |
|---|---|
| each hint | 0.50^n up to 3 hints |
| guided | 0.70 |
| heavily guided | 0.40 |
| transfer context | 1.30 |
| delayed recall | 1.50 |
| same-day repeat | 0.60 |
| guessed-but-correct | 0.50 |

Final weight MUST be clamped to [0.05, 2.00].

## 9.2 Mastery update

Mastery MUST use EMA-style smoothing and MUST NOT upgrade mastery bands until a minimum evidence floor is met.

```
pub const EMA_ALPHA: f64 = 0.30;
pub const MIN_ATTEMPTS_FOR_BAND_CHANGE: u32 = 3;
```

```
new_mastery = alpha * effective_input + (1 - alpha) * old_mastery
```

**Rules:**

- one correct answer never upgrades a band
- band changes require at least 3 attempts and meaningful spread
- readiness claims require a much stronger evidence contract than ordinary band changes

## 9.3 Weakness score

Retain Agent 1's useful decomposition:

```
Weakness =
  0.35 * mastery_deficit
+ 0.20 * link_breakage
+ 0.15 * misconception_pressure
+ 0.10 * representation_gap
+ 0.10 * timed_gap
+ 0.05 * guess_penalty
+ 0.05 * recency_decay
```

Use it to rank:

- practice candidates
- repair candidates
- mock candidates

- daily plan focus

## 9.4 Gap priority score

```
GapPriority =
   0.30 * dependency_block
 + 0.20 * recency_decay
 + 0.15 * exam_weight
 + 0.15 * misconception_density
 + 0.10 * repair_effort
 + 0.10 * confidence_gap
```

## 9.5 Forecast score

Retain Agent 1's forecast formula, because it is the most concrete instantiation of the exam-intelligence material.

```
ForecastScore =
   0.25 * frequency
 + 0.20 * recency
 + 0.15 * trend
 + 0.15 * bundle_strength
 + 0.10 * syllabus_priority
 + 0.10 * style_regime_fit
 + 0.05 * examiner_goal_fit
```

Bands:

- High: >= 7000
- Medium: 4500..6999
- SurpriseRisk: 3000..4499
- Uncertain: < 3000

## 9.6 Mock orchestration and selection

Two levels are retained:

### Topic-level mock orchestration

```
MockOrchestration =
   0.25 * weakness
 + 0.20 * coverage_gap
 + 0.20 * misconception_pressure
 + 0.15 * spaced_due
 + 0.10 * exam_weight
 + 0.10 * info_value
 + 0.05 * variety_bonus
 - 0.25 * anti_repeat_penalty
```

### Question-level selection

```
MockSelection =
   0.30 * blueprint_fit
 + 0.20 * diagnostic_need
 + 0.15 * coverage_need
 + 0.10 * info_value
 + 0.10 * representation_need
 + 0.10 * variety
 + 0.05 * surprise_risk
 - 0.25 * anti_repeat
```

## 9.7 Readiness score

The canonical runtime readiness formula is:

```
Readiness =
   0.45 * mastery
 + 0.20 * timed_performance
 + 0.15 * coverage
 + 0.10 * consistency
 + 0.10 * trend
 - penalties
```

### Penalties

- critical-topic failures
- recurring mistakes
- inactivity
- exam-anxiety / pressure collapse
- recent decline

### Readiness proof contract

A full readiness claim MUST verify all of the following:

| Requirement | Threshold |
|---|---:|
| attempts per topic | >= 15 |
| distinct sessions per topic | >= 3 |
| timed attempts per topic | >= 5 |
| full mocks completed | >= 1 |
| coverage at Stable-or-better | >= 80% of required topics |
| exam-critical topics below 5500 | none allowed |
| unresolved critical danger zones | none allowed |

## 9.8 Memory decay

The final core memory decay function is:

MemoryStrength(t) = initial_strength * e^(-decay_rate * days_since_last_review)

### Suggested rates

| State | Decay rate / day |
|---|---|
| Fresh | 0.05 |
| Consolidating | 0.08 |
| Stable | 0.03 |
| AtRisk | 0.15 |
| Decaying | 0.25 |
| Critical | 0.40 |
| Forgotten | 0.60 |

These SHOULD be configurable in the threshold registry.

## 9.9 Elite performance score

Elite remains a policy overlay on the same truth model.

```
EPS =
  w1 * speed
 + w2 * accuracy_under_pressure
 + w3 * transfer_ability
 + w4 * novelty_handling
 + w5 * misconception_immunity
```

Default subject-agnostic weights MAY be equal, but weights SHOULD be registry-driven.

### Elite entry gate

- entitlement MUST be Elite
- at least 70% of enrolled topics MUST be ExamReady or better
- self-select calibration mode MAY exist, but it MUST NOT bypass entitlement rules for locked features

## 9.10 Rise stage gates

Retain Agent 1's gate logic, but treat it as a distinct transformation engine rather than a generic mastery ladder.

| Transition | Gate |
|---|---|
| Rescue -> Stabilize | foundation >= 4500 and misconception_density < 3500 |

| Transition | Gate |
|---|---|
| Stabilize -> Accelerate | recall >= 6000 and accuracy >= 6500 |
| Accelerate -> Dominate | speed >= 7500 and pressure_stability >= 7000 |

## 10. Command surface

The command layer SHOULD remain DTO-only and thin. Domain entities MUST NOT be returned raw to the frontend.

### 10.1 Runtime command groups

| Group | Core commands |
|---|---|
| identity | create_local_account, authenticate_with_pin, switch_role_context, update_account_profile |
| packs | install_content_pack, list_installed_packs, get_pack_status |
| coach | resolve_next_coach_action, get_mission_stack, generate_daily_plan, acknowledge_mission_review |
| sessions | start_session, submit_attempt, pause_session, resume_session, complete_session, abandon_session |
| diagnostics | launch_diagnostic, get_diagnostic_report, get_problem_cards |
| repair | generate_repair_plan, start_gap_repair, submit_gap_repair_attempt, close_gap_repair |
| memory | get_review_queue, record_retrieval_attempt, start_memory_rescue |
| readiness/reporting | get_readiness_report, evaluate_readiness_claim, generate_parent_digest |
| library/glossary | get_library_shelves, save_library_item, search_glossary, record_glossary_event, start_glossary_audio_program |
| admin | export_local_backup, restore_local_backup, get_system_status, |

| Group | Core commands |
|---|---|
| | run_migration_check |

## 10.2 Foundry command groups

| Group | Core commands |
|---|---|
| source intake | upload_source_document, start_parse_job, get_source_pipeline_status |
| curriculum ops | review_parsed_curriculum, publish_curriculum_version, get_curriculum_diff |
| artifact ops | start_artifact_build, review_artifact, approve_artifact_for_publish, get_artifact_comparison |
| packaging | publish_pack_version, rollback_pack_version, get_pack_manifest, get_publish_history |
| evaluation | run_quality_evaluation, get_topic_health |

## 10.3 Canonical runtime DTOs

### Account creation

```rust
pub struct CreateLocalAccountInput {
    pub account_type: Role,          // Student | Parent | LocalAdmin
    pub display_name: String,
    pub pin: String,
    pub avatar_path: Option<String>,
    pub entitlement_tier: Option<EntitlementTier>,
}

pub struct AccountDto {
    pub id: i64,
    pub account_type: Role,
    pub display_name: String,
    pub avatar_path: Option<String>,
    pub entitlement_tier: EntitlementTier,
    pub first_run: bool,
    pub created_at: String,
}
```

### Pack installation

```rust
pub struct InstallContentPackInput {
    pub pack_path: String,
}

pub struct ContentPackInstallResultDto {
    pub pack_id: String,
    pub version: String,
    pub status: String,
    pub subjects_installed: Vec<String>,
    pub installed_at: String,
}
```

### Session start

```rust
pub struct StartSessionInput {
    pub account_id: i64,
    pub session_type: String,
    pub subject_id: Option<i64>,
    pub topic_scope: Vec<i64>,
    pub question_count: Option<u32>,
    pub duration_minutes: Option<u32>,
    pub is_timed: bool,
}

pub struct SessionHandleDto {
    pub session_id: i64,
    pub session_type: String,
    pub status: String,
    pub first_question: QuestionDto,
    pub remaining_count: u32,
    pub started_at: String,
}
```

### Attempt submission

```rust
pub struct SubmitAttemptInput {
    pub session_id: i64,
    pub question_id: i64,
    pub selected_option_id: Option<i64>,
    pub answer_text: Option<String>,
    pub confidence_level: Option<String>,
    pub hint_count: u32,
    pub changed_answer_count: u32,
    pub response_time_ms: Option<u32>,
}
```

```rust
pub struct AttemptFeedbackDto {
    pub is_correct: bool,
    pub explanation: Option<String>,
    pub misconception_tags: Vec<String>,
    pub evidence_weight_bp: BasisPoints,
    pub mastery_delta_bp: i32,
    pub next_question: Option<QuestionDto>,
    pub session_complete: bool,
}
```

### Diagnostic launch

```rust
pub struct LaunchDiagnosticInput {
    pub account_id: i64,
    pub subject_id: i64,
    pub mode: String, // quick | standard | deep
}

pub struct DiagnosticRunDto {
    pub diagnostic_id: i64,
    pub status: String,
    pub current_phase: String,
    pub first_question: QuestionDto,
}
```

### Next coach action

```rust
pub struct NextCoachActionDto {
    pub journey_state: String,
    pub operational_mode: String,
    pub action_type: String,
    pub title: String,
    pub reason: String,
    pub target_subject_id: Option<i64>,
    pub target_topic_ids: Vec<i64>,
    pub blocking_reasons: Vec<String>,
}
```

### Parent digest

```rust
pub struct ParentDigestDto {
    pub student_id: i64,
    pub readiness_bp: BasisPoints,
    pub risk_level: String,
    pub summary_text: String,
    pub active_alerts: Vec<String>,
    pub recommended_actions: Vec<String>,
```

```
    pub generated_at: String,
}
```

## 10.4 Command boundary rules

- commands MUST return DTOs, not DB rows
- commands MUST map domain errors to a unified CommandError
- every mutation command MUST emit at least one domain event
- long-running work SHOULD return job handles rather than blocking UI
- frontend MUST NOT compose academic decisions from multiple low-level endpoints if a single backend read model is appropriate

## 10.5 Canonical Foundry DTOs

### Source upload

```
pub struct UploadSourceDocumentInput {
    pub document_type: String,
    pub subject_id: Option<i64>,
    pub source_description: Option<String>,
    pub file_path: String,
}

pub struct SourceDocumentDto {
    pub source_document_id: i64,
    pub document_key: String,
    pub document_type: String,
    pub processing_status: String,
    pub uploaded_at: String,
}
```

### Publish pack version

```
pub struct PublishPackVersionInput {
    pub curriculum_version_id: i64,
    pub subject_ids: Vec<i64>,
    pub release_channel: String,
    pub version_label: String,
    pub notes: Option<String>,
}

pub struct PublishPackVersionResultDto {
    pub publish_version_id: i64,
    pub pack_id: String,
    pub version_label: String,
    pub release_channel: String,
    pub manifest_uri: String,
```

```
    pub status: String,
    pub published_at: String,
}
```

#### Artifact review

```
pub struct ReviewArtifactInput {
    pub artifact_version_id: i64,
    pub decision: String, // approve | reject | request_changes
    pub notes: Option<String>,
}

pub struct ArtifactReviewResultDto {
    pub artifact_version_id: i64,
    pub review_status: String,
    pub reviewed_by: String,
    pub reviewed_at: String,
}
```

# 11. Search, retrieval, and ranking

## 11.1 Runtime search

Runtime search MUST support:

- glossary search
- library search
- pack-local content retrieval
- topic-aware question retrieval
- weakness-aware ranking

SQLite FTS SHOULD power glossary/library search.

## 11.2 Foundry retrieval

Foundry retrieval MUST be curriculum-aware and provenance-aware.

Ranking SHOULD consider:

- exact/alias match
- curriculum context
- topic/skill linkage
- exam relevance
- source trust
- recency/frequency for source evidence

### 11.3 Retrieval truth rules

- runtime may only use installed pack content as truth
- foundry may use broader retrieval during build/review workflows
- externally retrieved material MUST NOT become live content without provenance, review, and publish steps

## 12. Roles, permissions, and security

### 12.1 Roles

| Role | Plane | Permissions |
|---|---|---|
| Student | runtime | own sessions, plans, library, diagnostics |
| Parent | runtime | child summaries, readiness, risks, parent-facing actions |
| LocalAdmin | runtime | device/account management, pack install, backup/restore |
| SuperAdmin | foundry | curriculum/content operations, publish, trust/policy administration |

### 12.2 Authentication and secrets

- local PINs MUST use argon2id
- lockout SHOULD trigger after 5 failed attempts
- exports SHOULD be encrypted
- sensitive local snapshots SHOULD be encrypted at rest when feasible
- pack manifests MUST be signed
- publish audit trail MUST be immutable

### 12.3 Privacy boundaries

- parent views SHOULD be plain-language summaries by default
- raw diagnostic internals SHOULD remain student/coach/internal only unless explicitly designed otherwise
- student uploads reused beyond the personal vault MUST require explicit consent/policy approval

### 12.4 Backup and restore rules

Runtime backup MUST include:

- SQLite database
- installed pack registry metadata
- threshold registry snapshot
- active feature flags

- pack activation pointers
- local search index version metadata

Runtime backup SHOULD NOT include large pack payload files by default unless the backup mode is explicitly full-device export.

Restore MUST:

1. validate backup version compatibility
2. verify integrity checksums
3. restore DB into staging
4. verify pack references
5. rebuild derived read models if needed
6. atomically swap to restored state

# 13. Observability, audit, and governance

## 13.1 Mandatory audit streams

The system MUST preserve:

- learner evidence history
- decision trace for major coach actions
- readiness claim trace
- intervention lifecycle trace
- pack install trace
- content provenance and publish history
- administrative changes

## 13.2 Explainability payload

For every major decision, store:

- evidence used
- gates evaluated
- hypotheses considered
- thresholds applied
- what was rejected
- final verdict
- trace ID

## 13.3 Shared policy module

A central platform-policy module SHALL own:

- mastery gates

- readiness contracts
- content publish gates
- entitlements and feature flags
- time-counting rules
- repair and blocking rules

# 14. Testing and evaluation strategy

## 14.1 Unit tests

Must cover:

- reducers
- formulas
- threshold gates
- planner precedence
- policy evaluators
- pack manifest validation

## 14.2 Integration tests

Must cover end-to-end flows:

1. attempt submission -> learner truth recompute
2. session completion -> plan adherence + next action
3. diagnostic run -> problem cards + repair plan
4. retrieval attempt -> memory state + review queue
5. publish pack -> install pack -> runtime queryable content

## 14.3 Replay tests

Every critical engine MUST support replay validation:

- same event stream => same learner state
- same event stream => same next action
- same event stream => same readiness verdict unless a threshold registry change is intentional and versioned

## 14.4 Contract tests

Must validate:

- runtime command DTO schemas
- foundry API contracts
- pack schema
- internal event schemas

- glossary/audio program schemas

### 14.5 Performance tests

Track at least:

| Concern | Target |
|---|---|
| submit_attempt hot-path latency | low double-digit ms on common local workloads |
| get_home_read_model | under 100 ms for normal local datasets |
| pack install | predictable and bounded by pack size |
| replay rebuild | deterministic and measurable |
| SQLite contention | acceptable under bursty attempt capture |

## 15. Delivery phases and vertical slices

This plan keeps Agent 2's phase logic but makes the exit criteria more implementation-concrete.

### Phase 0 - Freeze canonical truth

**Deliverables**

- canonical glossary of entities and enums
- ownership map
- threshold registry v1
- event naming spec
- pack schema v1
- migration skeletons
- conflict-resolution appendix approved

**Exit criteria**

- no open ambiguity on CurriculumNode vs ConceptAtom vs SkillAtom
- no open ambiguity on journey state vs coach mode
- no open ambiguity on mastery thresholds or entitlements

### Phase 1 - Runtime foundation

**Deliverables**

- runtime workspace
- SQLite migration system
- auth/PIN
- event ledger

- pack install + manifest verification
- backup/restore v1

**Exit criteria**

- one pack installs locally
- one learner account can authenticate and query installed content
- one trivial session can persist events

## Phase 2 - Curriculum, knowledge, and question substrate

**Deliverables**

- curriculum graph install tables
- knowledge/glossary/formula install tables
- question intelligence substrate
- question family and misconception mappings
- FTS foundations

**Exit criteria**

- one subject is fully installable and queryable by node, concept, skill, question family, and glossary entry

## Phase 3 - Learner truth engine

**Deliverables**

- skill/concept/node reducers
- learner snapshots
- error and pressure slices
- recompute pipeline

**Exit criteria**

- attempts deterministically update learner truth and survive replay

## Phase 4 - Coach core and session runtime

**Deliverables**

- learner journey state machine
- coach operational mode state machine
- next-action resolver
- session runtime
- counted-time truth

**Exit criteria**

- backend can decide what the learner should do next and explain why

## Phase 5 - Diagnostics and repair

**Deliverables**

- multi-session diagnostic engine
- one-session DNA diagnostic
- problem cards
- hypothesis/probe pipeline
- repair plan generation

**Exit criteria**

- defensible problem-cause-fix report for one supported subject

## Phase 6 - Memory, planning, and readiness

**Deliverables**

- decay/review engine
- daily/weekly planning
- readiness claims/proofs
- parent digest v1

**Exit criteria**

- backend can generate daily missions, review queues, and readiness outputs from real evidence

## Phase 7 - Library, glossary, and traps

**Deliverables**

- typed library shelves
- glossary lab backend
- knowledge bundles
- question-glossary links
- traps/contrast backend
- glossary audio program queue

**Exit criteria**

- learner can move from a mistake -> linked glossary/trap recovery -> back into practice

## Phase 8 - Foundry and publishing

**Deliverables**

- source intake
- OCR/layout parse

- curriculum ingestion/review/publish
- artifact build/compare/quality gate
- pack compile/sign/release

**Exit criteria**

- admin can ingest sources, review outputs, publish a pack, and runtime can install it safely

### Phase 9 - Hardening and calibration

**Deliverables**

- replay harness
- shadow decision evaluation
- migration hardening
- backup/restore hardening
- audit viewers
- threshold calibration tooling

**Exit criteria**

- every critical engine has replay coverage and release gates

### 15.1 Immediate build order

1. freeze canonical entities, thresholds, state machines, ownership
2. implement runtime schema and migration framework
3. implement auth, roles, and pack install
4. implement curriculum + one subject content pack
5. implement question + skill linkage
6. implement attempt ledger + learner truth recompute
7. implement coach state + next action
8. implement session runtime
9. implement diagnostics
10. implement memory + readiness
11. implement library/glossary/traps
12. implement foundry
13. harden replay, audits, migrations, and calibration

## 16. Decisions that explicitly resolve source-plan conflicts

| Conflict | Final resolution |
|---|---|
| BasisPoints as u16 vs i32 | u16 for persisted score domain; signed types only where mathematically needed |
| one coach state machine vs two meanings | split into LearnerJourneyState and |

| Conflict | Final resolution |
|---|---|
| | CoachOperationalMode |
| Functional at 4000 vs 5500 | Functional starts at 4000; Stable starts at 5500; readiness coverage uses Stable-or-better |
| EntitlementTier drift | Standard, Premium, Elite only |
| mixed memory ladders | one core scheduler ladder; UI/display labels are derived |
| runtime vs foundry blur | raw source/OCR/review/publish live only in Foundry |
| premium/elite as separate architectures | policy layers on shared learner truth |
| monolithic 22 local migrations including intake | runtime keeps install/runtime truth; foundry has its own migration track |

## 17. Risks to manage early

1. **Canonical unit drift**
   If question-to-skill mapping is weak, everything downstream degrades.

2. **Threshold sprawl**
   Readiness, memory, diagnostics, and planning all need governed calibration.

3. **Publish-trust failure**
   OCR noise or unreviewed artifacts must never leak into live packs.

4. **Frontend logic leakage**
   The UI must not re-implement backend reasoning.

5. **Migration complexity**
   Pack versions, schema versions, and curriculum versions will evolve independently.

6. **Scope explosion**
   Many idea files describe a full platform future. Delivery must stay vertical-slice first.

## 18. What not to build first

Do **not** prioritize the following before the core truth and runtime are stable:

- multiplayer game backends
- large-scale sync of full learner state
- broad concierge automation
- unconstrained LLM generation as truth
- every audio/glossary innovation
- optimization/RL loops over unstable deterministic baselines

## 19. Final recommendation

The correct implementation shape is:

- **local modular runtime**
- **central content operating system**
- **deterministic evented coaching core**
- **shared curriculum/question/knowledge substrate**
- **audited readiness-and-proof model**
- **premium and elite as overlays, not forks**

That is the backend shape that best preserves Agent 2's architectural correctness while retaining Agent 1's implementability.

## Appendix A - Canonical thresholds registry (initial)

| Key | Value |
|---|---|
| PIN_MAX_ATTEMPTS | 5 |
| PIN_LOCKOUT_MINUTES | 5 |
| SESSION_MAX_DURATION_MINUTES | 90 |
| MIN_ATTEMPTS_FOR_BAND_CHANGE | 3 |
| MASTERY_PROTECTION_GATE | 4000 |
| MASTERY_DEPENDENCY_UNLOCK_GATE | 5500 |
| MASTERY_EXAM_READY_MIN | 8500 |
| READINESS_COVERAGE_MIN_PERCENT | 0.80 |
| READINESS_MIN_MOCK_COUNT | 1 |
| EXAM_MODE_TRIGGER_DAYS | 14 |
| INACTIVITY_DANGER_DAYS | 3 |
| ELITE_TOPIC_COVERAGE_PERCENT | 0.70 |

## Appendix B - Sample ownership map

| Table family | Owning module |
|---|---|
| student_question_attempts, session_events | domain-session-runtime |
| skill_states, concept_states, curriculum_node_states | domain-learner-truth |

| Table family | Owning module |
|---|---|
| memory_states, review_schedules, interference_edges | domain-memory |
| diagnostic_cases, problem_cards | domain-diagnostics |
| coach_plans, coach_missions, learner_journey_states, coach_operational_modes | domain-planning |
| readiness_claims, danger_zones, readiness_reports | domain-readiness |
| library_items, mistake_bank_entries, revision_pack_items | domain-library |
| source_documents, ocr_results, artifact_versions, publish_versions | Foundry modules |

## Appendix C - Minimal runtime command error model

```rust
#[derive(Debug, Serialize)]
#[serde(tag = "type", content = "message")]
pub enum CommandError {
  NotFound(String),
  Unauthorized(String),
  ValidationError(String),
  BusinessRuleViolation(String),
  StateConflict(String),
  DatabaseError(String),
  InternalError(String),
}
```

## Appendix D - One-line build doctrine

**Freeze the truth model first, then build the runtime vertical slice, then add the diagnostic/memory/planning intelligence, then build Foundry, then harden replay and governance.**